

DericBI Stock Intelligence — Full System Setup Guide

Stack: Python FastAPI (Hugging Face Spaces) + React/Vite (Vercel)

Time to deploy: ~45 minutes end-to-end

Table of Contents

1. Repo Structure
 2. Prerequisites
 3. Backend — Hugging Face Spaces
 4. Frontend — Vercel
 5. Connecting Frontend Backend
 6. Local Development
 7. Verifying Everything Works
 8. Troubleshooting
 9. Extending the System
-

1. Repo Structure

```
dericbi-stock-intelligence/  
  backend/                                ← Hugging Face Space  
    app.py  
    requirements.txt  
    data/  
      portfolio_trades.csv  
      prices_history.csv  
      fundamentals.csv  
      config.json  
    services/  
      __init__.py  
      data_loader.py  
      scoring.py  
      portfolio.py  
      analytics.py  
    utils/  
      __init__.py  
      helpers.py  
  
  frontend/                               ← Vercel app  
    index.html  
    vite.config.js
```

```
package.json
.env.example
src/
  main.jsx
  App.jsx
```

2. Prerequisites

Install these on your local machine before starting:

Tool	Install
Git	https://git-scm.com
Node.js 18+	https://nodejs.org
Python 3.10+	https://python.org
A GitHub account	https://github.com
A Hugging Face account	https://huggingface.co
A Vercel account	https://vercel.com (free, sign in with GitHub)

3. Backend — Hugging Face Spaces

Step 3.1 — Create the GitHub repo

```
# On your local machine
git init dericbi-stock-intelligence
cd dericbi-stock-intelligence

# Copy the backend/ folder you downloaded into this repo
# Then commit everything
git add .
git commit -m "Initial backend commit"

# Push to GitHub (create the repo on GitHub first, then)
git remote add origin https://github.com/YOUR_USERNAME/dericbi-stock-intelligence.git
git push -u origin main
```

Step 3.2 — Create a Hugging Face Space

1. Go to <https://huggingface.co/new-space>
2. Fill in:
 - **Space name:** dericbi-backend (or any name you like)
 - **SDK:** Select “**Docker**” ← important
 - **Visibility:** Public (free tier requires public)
3. Click **Create Space**

Step 3.3 — Add a Dockerfile

In the root of your Space repository (you can edit directly on HF), create a file called `Dockerfile`:

```
FROM python:3.11-slim

WORKDIR /app

# Install dependencies
COPY backend/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy all backend files
COPY backend/ .

# Create data directory (persists CSV files)
RUN mkdir -p data

# Expose the port Spaces expects
EXPOSE 7860

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "7860"]
```

Alternative (no Docker): Use the **Gradio** SDK and add a `app.py` that runs FastAPI with `uvicorn` — but Docker is cleaner.

Step 3.4 — Connect GitHub to Spaces (auto-deploy)

In your Space settings → **Repository** → link to your GitHub repo → select the branch `main`.

Every `git push` will now auto-rebuild and redeploy your backend.

Step 3.5 — Verify the backend is live

Once the Space finishes building (takes 2–5 minutes), open:

`https://YOUR_USERNAME-dericbi-backend.hf.space/`

You should see:

```
{"message": "DericBI Stock Intelligence API", "status": "running"}
```

Test an endpoint:

`https://YOUR_USERNAME-dericbi-backend.hf.space/api/stocks?timing=best_pick`

Note down your Space URL — you'll need it for the frontend.

4. Frontend — Vercel

Step 4.1 — Install dependencies locally

```
cd frontend/  
npm install
```

Step 4.2 — Configure the backend URL

```
# Copy the example env file  
cp .env.example .env.local  
  
# Edit .env.local with your actual Spaces URL  
# VITE_API_BASE=https://YOUR_USERNAME-dericbi-backend.hf.space
```

Step 4.3 — Test locally first

```
npm run dev  
# Opens at http://localhost:5173
```

The app will show demo/mock data if the backend is unreachable locally, and live data when connected.

Step 4.4 — Deploy to Vercel

Option A — Vercel CLI (fastest):

```
npm install -g vercel  
vercel login  
vercel --prod  
# Follow the prompts - Vercel auto-detects Vite
```

Option B — Vercel Dashboard: 1. Go to <https://vercel.com/new> 2. Click “**Import Git Repository**” 3. Select your GitHub repo → choose the `frontend/` folder as root 4. Vercel auto-detects Vite/React — click **Deploy**

Step 4.5 — Set environment variable in Vercel

In Vercel dashboard → Your Project → **Settings** → **Environment Variables**:

Key	Value
VITE_API_BASE	https://YOUR_USERNAME-dericbi-backend.hf.space

After setting, trigger a **Redeploy** from the Deployments tab.

5. Connecting Frontend Backend

The single connection point is the `VITE_API_BASE` environment variable. The frontend's `App.jsx` reads it at the top:

```
const API_BASE = import.meta.env.VITE_API_BASE || "https://your-space.hf.space";
```

All 5 API calls map like this:

Frontend action	API call
Load home dashboard	GET /api/stocks?timing=best_pick
Open stock screener	GET /api/stocks?timing=daily (etc.)
View stock detail	GET /api/stock/EQTY.NR
View portfolio	GET /api/portfolio
Log a trade	POST /api/trades
View analytics	GET /api/analytics

CORS is already configured

The backend's `app.py` includes:

```
app.add_middleware(CORSMiddleware, allow_origins=["*"], ...)
```

So the browser won't block requests from your Vercel domain.

6. Local Development

Run both services simultaneously for full local development:

Terminal 1 — Backend:

```
cd backend/  
pip install -r requirements.txt  
uvicorn app:app --reload --port 8000  
# API runs at http://localhost:8000
```

Terminal 2 — Frontend:

```
cd frontend/  
# Set .env.local: VITE_API_BASE=http://localhost:8000  
npm run dev  
# App runs at http://localhost:5173
```

7. Verifying Everything Works

Run through this checklist after deploying:

```
[ ] Backend root URL returns {"status": "running"}
[ ] /api/stocks returns a list of stocks with scores
[ ] /api/portfolio returns summary + holdings (empty is fine)
[ ] POST /api/trades with test data works (try with curl or Postman)
[ ] /api/analytics returns equity_curve array
[ ] Frontend home page loads stocks from backend (not mock data)
[ ] Screener pill filters change the sort order
[ ] Tapping a stock opens the detail page with real price chart
[ ] "Simulate Buy" opens modal, submitting logs the trade
[ ] Portfolio page shows the logged trade
[ ] Analytics page shows charts and projections
[ ] CSV export downloads a file
[ ] Footer link to dericbi.vercel.app is visible
```

Test a trade with curl

```
curl -X POST https://YOUR_USERNAME-dericbi-backend.hf.space/api/trades \
-H "Content-Type: application/json" \
-d '{
  "ticker": "EQTY.NR",
  "trade_type": "BUY",
  "quantity": 500,
  "price": 38.0,
  "date": "2025-01-15"
}'
```

8. Troubleshooting

Backend not responding

- Check the Hugging Face Space logs (Space → Logs tab)
- Make sure the Dockerfile EXPOSE port matches CMD port (both 7860)
- Free Spaces sleep after inactivity — first request takes ~15s to wake up
- Add a health check ping on frontend load to wake it early

Stocks returning empty / errors

- NSE tickers on Yahoo Finance use .NR suffix (e.g. EQTY.NR)
- Some tickers may be delisted or temporarily unavailable — `data_loader.py` catches errors per ticker
- Check `yfinance` version compatibility: pin to `yfinance==0.2.40` in `requirements.txt`

CORS errors in browser

- Confirm `allow_origins=["*"]` is in `app.py`

- Confirm VITE_API_BASE does NOT have a trailing slash
- Try the API URL directly in browser to confirm it's reachable

Portfolio CSV not persisting on Hugging Face

- Free tier Spaces are **ephemeral** — the filesystem resets on redeploy
- For persistence, use Hugging Face's **Dataset** storage or upgrade to a persistent Space
- Workaround: Export trades CSV often using the frontend download button, re-import manually

Frontend shows only mock data

- Check browser console for fetch errors
- Verify VITE_API_BASE is set correctly in Vercel environment variables
- Confirm you redeployed after setting the env variable

9. Extending the System

Add a new NSE ticker

In backend/app.py, add to the NSE_TICKERS list:

```
{"ticker": "BAT.NR", "name": "BAT Kenya", "sector": "Consumer"},
```

Change scoring weights

Edit backend/data/config.json:

```
{
  "scoring_weights": {
    "daily": 0.5,
    "monthly": 0.3,
    "long_term": 0.2
  }
}
```

Then update `scoring.py` to read from config instead of hardcoded weights.

Add price alerts

1. Backend: add GET `/api/alerts` and POST `/api/alerts` endpoints
2. Store alerts in `data/alerts.csv`
3. Frontend: add an Alerts tab in the bottom nav

Add push notifications (PWA)

1. Add a `manifest.json` and service worker to `frontend/public/`
2. Register the service worker in `main.jsx`
3. Use the Web Push API for price threshold alerts

Summary

GitHub Repo

backend/	auto-deploys to	Hugging Face Space (FastAPI + yfinance)
frontend/	auto-deploys to	Vercel (React + Vite)
		calls API via VITE_API_BASE env var

Your live URLs will be: - Backend: `https://YOUR_USERNAME-dericbi-backend.hf.space`
- Frontend: `https://dericbi-stock-intelligence.vercel.app` (or custom domain)

DericBI Stock Intelligence — Know more. Decide better.
More BI services: <https://dericbi.vercel.app>